

UT_03.1_Páginas web que cambian dinámicamente

Hasta ahora, hemos utilizado el navegador como un medio para alcanzar un fin: ejecutar ejemplos relativamente sencillos. Pero el potencial del lenguaje dentro del navegador no se aprovecha al máximo hasta que se utiliza para crear una aplicación web dinámica. Una base importante para ello es el Modelo de Objetos de Documento, que gestiona la estructura de una página web en forma de árbol y puede modificarse dinámicamente mediante JavaScript.

En general, el contenido se puede generar de forma dinámica tanto en el *backend* (es decir, en el lado del servidor que envía el contenido de una página web al cliente) como en el *frontend*. En este tema nos centraremos en la creación de contenido de forma dinámica en el *frontend*, lo que significa generar contenido utilizando JavaScript.

1.1 Estructura de una página web

Ya sabes que la programación orientada a objetos intenta describir objetos del mundo real como objetos en el modelado de programas. Incluso una página web (sobre la que se puede discutir si pertenece al mundo real) se representa internamente en el navegador como un objeto.

1.1.1 Modelo de objetos de documento

Cada vez que accedes a una página web, el navegador crea un modelo correspondiente de la página web en la memoria, denominado *Modelo de objetos de documento (DOM)*. El DOM se utiliza principalmente para acceder al contenido de la página web a través de JavaScript para modificar el contenido existente o añadir nuevo contenido, por ejemplo. Representa los componentes de una página web jerárquicamente en una *representación en árbol*, también conocida como *árbol DOM*. Un árbol DOM, a su vez, se compone de *nodos*. Su disposición jerárquica refleja la estructura de una página web:

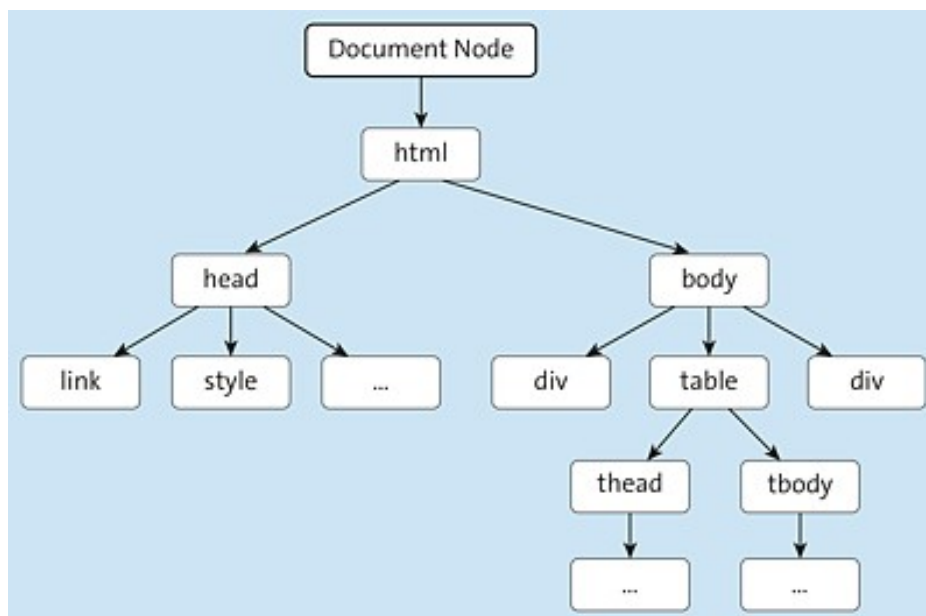


Figura 1.1 Estructura de un árbol DOM

Información de fondo

La representación en forma de árbol es una estructura de datos que se utiliza con frecuencia en informática y programación, especialmente cuando se trata de representar relaciones entre

partes y el todo. En el caso del DOM, el elemento inicial (la raíz) se encuentra en la parte superior y el árbol «crece» hacia abajo.

1.1.2 Los diferentes tipos de nodos

En total, hay cuatro tipos principales de nodos (hay algunos más, doce en total para ser exactos, pero ocho de ellos son menos relevantes en esta etapa). La mejor forma de explicarlos es con un ejemplo. **El Listado 1.1** muestra un ejemplo de archivo HTML en el que se puede ver el código HTML de una tabla sencilla para mostrar una lista de contactos. El DOM correspondiente se muestra en **la Figura 1.2** (aunque la ilustración no está completa por razones de espacio y claridad).

```
<!DOCTYPE html>
<html>
  <head lang="en">
    <title>Contacts Example</title>
  </head>
  <body>
    <main id="main">
      <h1>Contacts</h1>
      <table id="contact-list-table" summary="Contacts">
        <thead>
          <tr>
            <th id="table-header-first-name">First name</th>
            <th id="table-header-last-name">Last name</th>
            <th id="table-header-email">E-mail address</th>
          </tr>
        </thead>
        <tbody>
          <tr class="row odd">
            <td>John</td>
            <td>Doe</td>
            <td>john.doe@javascripthandbuch.de</td>
          </tr>
          <tr class="row even">
            <td>James</td>
            <td>Doe</td>
            <td>james.doe@javascripthandbuch.de</td>
          </tr>
          <tr class="row odd">
            <td>Peter</td>
            <td>Doe</td>
            <td>peter.doe@javascripthandbuch.de</td>
          </tr>
          <tr class="row even">
            <td>Paul</td>
            <td>Doe</td>
            <td>paul.doe@javascripthandbuch.de</td>
          </tr>
        </tbody>
      </table>
    </main>
  </body>
</html>
```

Listado 1.1 Ejemplo de página HTML

Cuando trabajes con el DOM, lo más probable es que utilices los siguientes cuatro tipos de nodos:

El **nodo del documento** (resaltado en negrita en **la Figura 1.2**) representa toda la página web y es la raíz del árbol DOM. Está representado por el objeto global del documento, que quizá ya hayas visto en algunos listados. Este objeto es también el objeto de entrada para cualquier trabajo con el DOM. El nodo del documento también se denomina *nodo raíz*.

Los **nodos de elemento** (mostrados en **la figura 1.2** con fondo blanco) representan elementos HTML individuales de una página web. En el ejemplo, estos son los elementos

<main>, <h1>, <table>, <thead> y <tbody>

por ejemplo.

Los **nodos de atributos** (que se muestran en **la figura 1.2** rodeados por líneas discontinuas y con fondo blanco) representan los atributos de los elementos HTML; en el ejemplo, los nodos de atributos para los atributos lang, id y summary.

El texto dentro de los elementos HTML se representa mediante su propio tipo de nodo, denominado *nodo de texto* (que se muestra en **la figura 1.2** rodeado de líneas discontinuas y coloreado en gris). En el ejemplo, se trata de los nodos de los textos Ejemplo de contactos, Contactos, Nombre, Apellidos y Dirección de correo electrónico. Los nodos de texto no pueden tener nodos secundarios y, por lo tanto, son inevitablemente hojas en el árbol DOM (en el ejemplo, no se muestran todos los nodos de texto debido a restricciones de espacio, como se mencionó anteriormente).

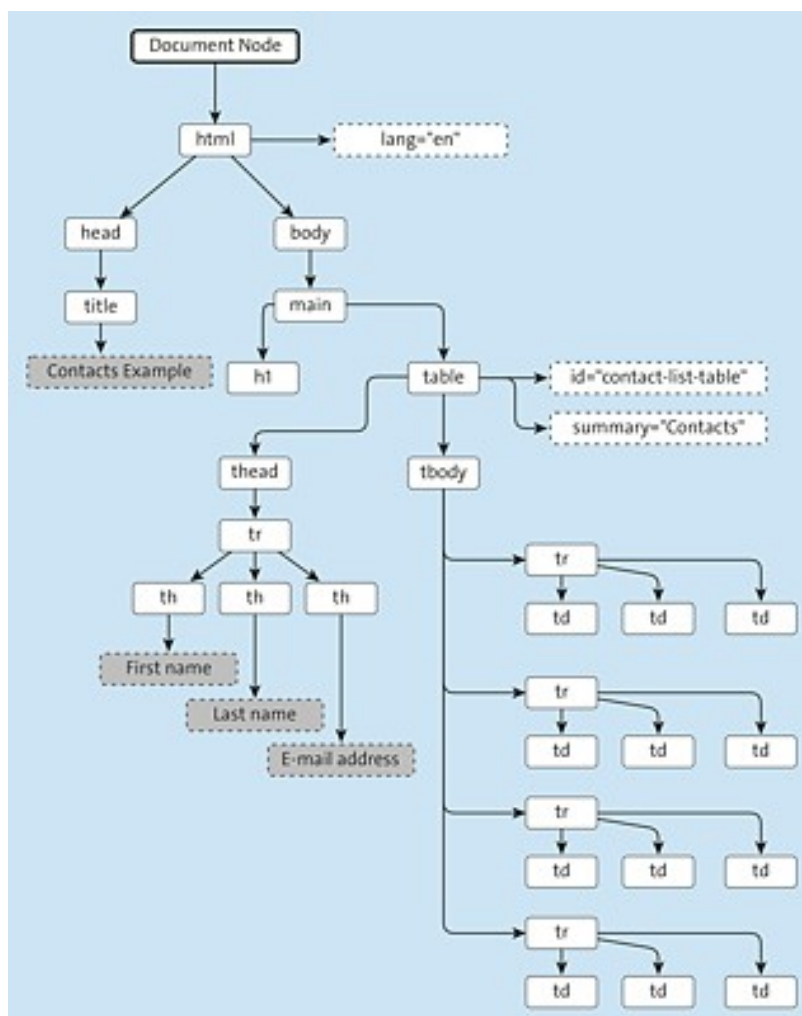


Figura 1.2 Ejemplo de estructura de árbol DOM

Nota

El ejemplo del **Listado 1.1** y **la Figura 1.2** constituyen la base de las siguientes secciones. Utilizando este ejemplo, mostraremos cómo acceder y modificar nodos en una página web.

Examen del DOM en el navegador

Puede ver el DOM de una página web en una vista especial con las respectivas herramientas de depuración de JavaScript de los diferentes navegadores. En Chrome DevTools, esta vista se encuentra en la pestaña **Elementos** (véase **la figura 1.3**).

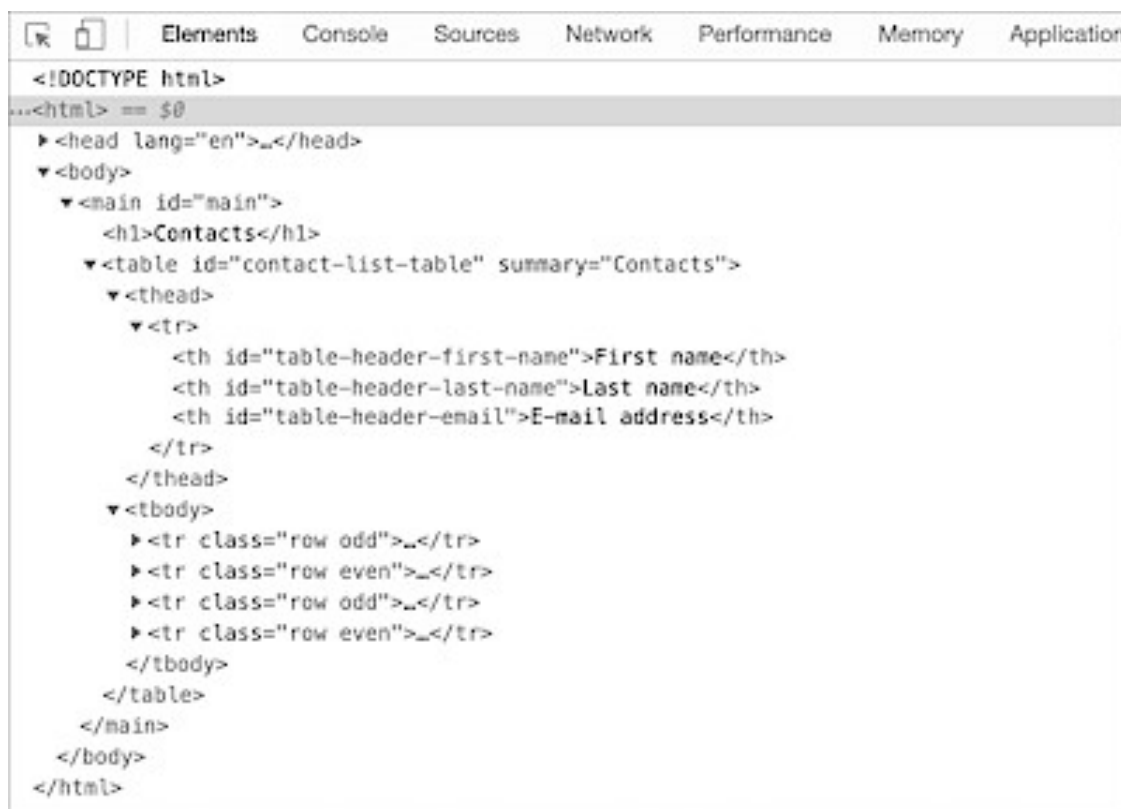


Figura 1.3 Representación del DOM en Chrome DevTools

Por lo general, incluso puede cambiar el DOM manualmente a través de esta vista. Pruébelo haciendo doble clic en uno de los nodos del árbol DOM, por ejemplo, un nodo de texto. A continuación, puede cambiar el texto correspondiente del nodo.

En la práctica, esto puede resultar muy útil para probar rápidamente una determinada característica de HTML. Sin embargo, los cambios que realice en esta vista no afectan al archivo HTML subyacente. Si vuelve a cargar el archivo en el navegador, los cambios se perderán.

1.1.3 El nodo del documento

El nodo del documento, como se mencionó anteriormente, es el punto de entrada al DOM y está representado por el objeto global del `document`, que tiene varias propiedades y métodos.

Las propiedades seleccionadas se enumeran en **la Tabla 1.1**, mientras que los distintos métodos se tratarán en detalle a lo largo de este tema.

Propiedad	Descripción
<code>document.title</code>	Contiene el título del documento actual.

<code>document.lastModified</code>	Contiene la fecha en la que se modificó el documento por última vez
<code>document.URL</code>	Contiene la URL del documento actual
<code>document.domain</code>	Contiene el dominio del documento actual.
<code>document.cookie</code>	Contiene una lista de todas las cookies del documento
<code>document.forms</code>	Contiene una lista de todos los formularios del documento
<code>document.images</code>	Contiene una lista de todas las imágenes del documento
<code>document.links</code>	Contiene una lista de todos los enlaces del documento

Tabla 1.1 Propiedades seleccionadas del objeto documento

DOM en Node.js

El DOM en forma de variable global `document` solo está disponible en entornos de tiempo de ejecución basados en navegador. Node.js, por ejemplo, no proporciona ninguna variable global de este tipo, ya que Node.js no se utiliza normalmente para renderizar páginas web. En Node.js, el DOM de una página web solo se puede crear mediante módulos especiales como `jsdom` (<https://github.com/jsdom/jsdom>), que se pueden utilizar para analizar páginas web.

La estructura del DOM (qué propiedades y métodos están disponibles, qué tipos de nodos se proporcionan, etc.) se define en la interfaz de programación de aplicaciones (API) del DOM, una especificación del World Wide Web Consortium (W3C). Esta API es independiente del lenguaje de programación; es decir, hay implementaciones no solo para JavaScript, sino también para otros lenguajes de programación como Java o C++.

Interfaz, implementación y API

En la programación orientada a objetos, *las interfaces* se utilizan para definir los métodos que deben estar presentes en *las implementaciones* (es decir, las realizaciones concretas de la interfaz respectiva). Una *API* define un conjunto de interfaces proporcionadas por un sistema de software.

En consecuencia, la API DOM es un conjunto de interfaces que los navegadores proporcionan para trabajar con páginas web.

1.2 Selección de elementos

Tanto si desea cambiar la información existente en una página web como añadir nueva información, primero debe seleccionar un elemento de la página web que desee cambiar o al que desee adjuntar la nueva información. Para ello, la API DOM ofrece varias propiedades y métodos, como se muestra en **la Tabla 1.2**.

Como puede ver, hay algunos métodos que devuelven varios elementos y otros que devuelven elementos únicos. Veremos los detalles en las siguientes secciones.

Propiedad/Método	Descripción	Retorno	Sección
<code>getElementById()</code>	Selecciona un elemento basándose en un ID	Elemento único	Sección 1.2.1

<code>getElementsBy-ClassName()</code>	Selecciona elementos basándose en un nombre de clase.	Lista de elementos	Sección 1.2.2
<code>getElementsBy-TagName()</code>	Selecciona todos los elementos con el nombre de elemento especificado	Lista de elementos	Sección 1.2.3
<code>getElementsByName()</code>	Selecciona elementos por su nombre	Lista de elementos	Sección 1.2.4
<code>querySelector()</code>	Devuelve el primer elemento que coincide con un selector CSS dado	Elemento único	Sección 1.2.5
<code>querySelectorAll()</code>	Devuelve todos los elementos que coinciden con un selector CSS determinado.	Lista de elementos	Sección 1.2.5
<code>parentElement</code>	Devuelve el elemento padre de un nodo	Elemento único	Sección 1.2.6
<code>parentNode</code>	Devuelve el nodo padre de un nodo.	Nodo único	Sección 1.2.6
<code>previousElement-Sibling</code>	Devuelve el elemento hermano anterior de un nodo	Elemento único	Sección 1.2.8
<code>previousSibling</code>	Devuelve el nodo hermano anterior de un nodo.	Nodo único	Sección 1.2.8
<code>nextElementSibling</code>	Devuelve el elemento hermano siguiente de un nodo.	Elemento único	Sección 1.2.8
<code>nextSibling</code>	Devuelve el siguiente nodo hermano de un nodo.	Nodo único	Sección 1.2.8
<code>firstElementChild</code>	Devuelve el primer elemento hijo de un nodo.	Elemento único	Sección 1.2.7
<code>firstChild</code>	Devuelve el primer nodo secundario de un nodo.	Nodo único	Sección 1.2.7
<code>lastElementChild</code>	Devuelve el último elemento secundario de un nodo	Elemento único	Sección 1.2.7
<code>lastChild</code>	Devuelve el último nodo hijo de un nodo.	Nodo único	Sección 1.2.7
<code>childNodes</code>	Devuelve todos los nodos hijos de un nodo.	Lista de nodos	Sección 1.2.7
<code>children</code>	Devuelve todos los elementos secundarios de un nodo	Lista de elementos	Sección 1.2.7

Tabla 1.2 Los diferentes métodos y propiedades para seleccionar elementos

Métodos de selección

Los métodos y propiedades de selección no solo están disponibles para el nodo del documento, sino también para otros nodos.

1.2.1 Selección de elementos por ID

A los elementos de una página web se les puede asignar un ID (que es único en esa página web) mediante el atributo `id`. Este ID se puede utilizar en reglas CSS, y se puede seleccionar el elemento correspondiente mediante JavaScript utilizando el método `getElementById()` del objeto `document`. Basta con pasar el ID del elemento que se desea seleccionar al método en forma de cadena.

En el **Listado 1.2**, se selecciona el elemento con el ID principal (véase también la **Figura 1.4**) y se almacena en la variable `mainElement`. A continuación, se utiliza la propiedad `className` para cambiar el atributo de `class` del elemento al valor `border`, lo que en el ejemplo da como resultado que el elemento tenga un borde rojo con esquinas redondeadas (véase la **figura 1.5**; el ejemplo completo, incluido el código HTML y CSS, se puede encontrar en la zona de descargas del libro).

```
const mainElement = document.getElementById('main');  
mainElement.className = 'border';
```

Listado 1.2 Acceso a un elemento a través del ID

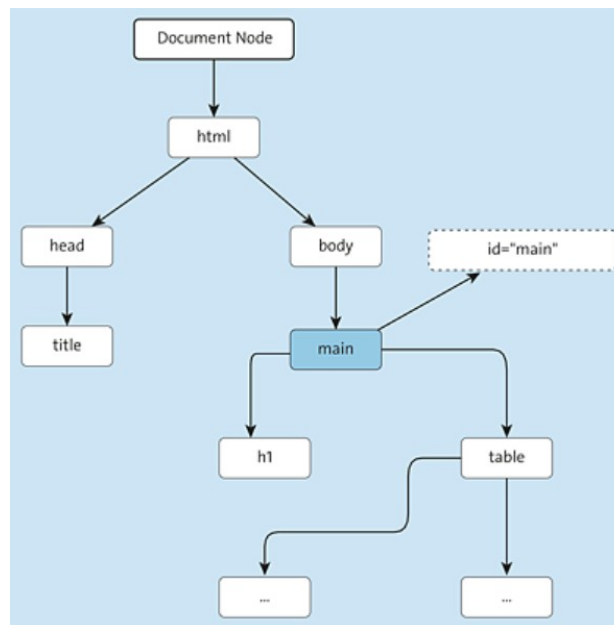


Figura 1.4 El método `getElementById()` selecciona como máximo un elemento

Contacts		
First name	Last name	E-mail address
John	Doe	john.doe@jascrisphandbuch.de
James	Doe	james.doe@jascrisphandbuch.de
Peter	Doe	peter.doe@jascrisphandbuch.de
Paul	Doe	paul.doe@jascrisphandbuch.de

Figura 1.5 Al elemento devuelto se le asigna una nueva clase CSS, lo que le da un borde resaltado

Consejo

En la práctica, no es mala idea programar de forma un poco más *defensiva* y comprobar que la variable a la que se va a acceder no es nula o indefinida. Esto también se aplica al trabajo con

el DOM, ya que el método `getElementById()` devuelve el valor nulo si no se encuentra ningún elemento con el ID pasado. Si luego intentas acceder a una propiedad o método del elemento supuesto, se produce un error de tiempo de ejecución. Para evitarlo, debes utilizar una consulta `if` para asegurarte de que el valor devuelto por `getElementById()` no es nulo, como se muestra en **el Listado 1.3**.

```
const mainElement = document.getElementById('main'); // Selecciona el
elemento por ID.
if(mainElement !== null) { // Si el elemento no está vacío,
  mainElement.className = 'border'; // asignar nueva clase CSS.
}
```

Listado 1.3 Si no hay ningún elemento con el ID principal, no se accede a la variable

Como alternativa, puede acortar el código y aprovechar el operador `&&`, donde el segundo operando solo se evalúa si el primero devuelve verdadero. Así, en **el Listado 1.4**, el operando `(mainElement.className = 'border')` solo se evalúa (o ejecuta) si `mainElement` tiene un valor que no se evalúa como falso, es decir, si no es nulo.

```
const mainElement = document.getElementById('main'); // Selecciona el
elemento por ID.
mainElement && (mainElement.className = 'border');
```

Listado 1.4 Se puede utilizar el operador `&&` para acortar esta comprobación

La comprobación es aún más corta si se utiliza el *operador de encadenamiento opcional* `?.`, disponible desde ES2020. Este operador solo accede a una propiedad o llama a un método cuando el valor de su izquierda no es `null` ni `undefined`; en caso contrario, la expresión devuelve `undefined` sin lanzar ningún error. ¡Atención!: el encadenamiento opcional sirve para leer propiedades y llamar a métodos, pero no puede usarse a la izquierda de una asignación (`mainElement?.className = 'border'` produce un `SyntaxError`). Por eso, en el Listado 1.5 se utiliza el método `classList.add()`, que añade la clase mediante una llamada a un método.

```
const mainElement = document.getElementById('main'); // Selecciona el
elemento por ID.
mainElement?.classList.add('border'); // Correcto
// mainElement?.className = 'border'; // SyntaxError
```

Listado 1.5 El operador `?.` permite leer propiedades o llamar a métodos de forma segura, pero no asignar valores

Rendimiento de los métodos de selección

En comparación con otros métodos de selección, seleccionar un elemento por ID es bastante rápido en términos de rendimiento, ya que una página web no puede tener varios elementos con el mismo ID y, por lo tanto, la búsqueda puede encontrar muy rápidamente el elemento adecuado para un ID. Otros métodos de selección, como el método `getElementsByClassName()` que se presenta en la siguiente sección, son mucho más lentos en comparación, ya que requieren comprobar todos los elementos de la página web. Normalmente no se nota la diferencia de velocidad, pero conviene tenerla en cuenta.

Consejo

No se deben abusar de los métodos DOM. Si necesitas utilizar el resultado de un método DOM en varios lugares dentro de un programa, almacena el resultado en una variable en lugar de llamar al método DOM una y otra vez. Ten en cuenta que cada llamada a un método DOM que busca elementos en el árbol DOM costará tiempo de cálculo. Este tiempo de cálculo se puede minimizar utilizando variables para almacenar los resultados en caché.

1.2.2 Selección de elementos por clase

Al igual que los ID, *las clases CSS* se pueden asignar a elementos individuales de una página web. Estas clases se gestionan a través del atributo `class`. Un elemento puede tener varias clases y, a diferencia de los ID, varios elementos pueden tener la misma clase.

Esto, a su vez, da como resultado que el método DOM `getElementsByClassName()` correspondiente, que permite la selección por clase CSS, devuelva no solo un único elemento, sino varios elementos, si es necesario.

Como argumento, al método se le pasa el nombre de la clase como una cadena, tal y como se muestra en **el Listado 1.6**. En este ejemplo, se seleccionan todos los elementos que contienen la clase CSS `even`, es decir, las dos filas pares de la tabla (véase **la Figura 1.6**).

```
const tableRowsEven = document
    .getElementsByClassName('even');    // Selecciona todas las filas pares
de la tabla.
```

Listado 1.6 Acceso a un elemento por nombre de clase

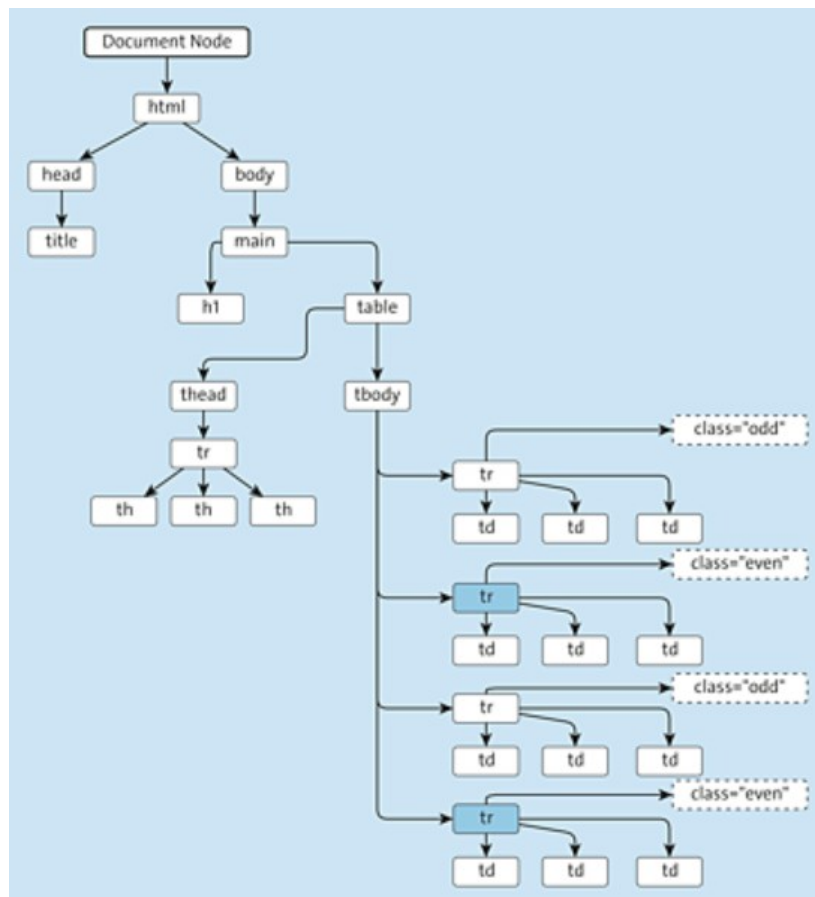


Figura 1.6 El método `getElementsByClassName()` puede devolver varios elementos

El valor de retorno de `getElementsByClassName()` es una colección de elementos (más precisamente, un objeto del tipo `HTMLCollection`), que se utiliza de forma similar a una matriz (pero que no es una matriz; más información al respecto en un momento). Existe otro tipo de colección muy parecido, `NodeList`, que es el que devuelven `querySelectorAll()`, `getElementsByName()` y la propiedad `childNodes`; ninguno de los dos es una matriz real. Esta colección contiene los elementos en el orden exacto en que aparecen en la página web.

Aunque estas colecciones parecen matrices a primera vista, no lo son. Los principiantes en JavaScript deben tener siempre presente este hecho, ya que su incumplimiento suele provocar errores en el programa.

Una característica que estas colecciones tienen en común con las matrices es que se puede acceder a los elementos individuales a través de un índice. Es decir, se puede utilizar `tableRowsEven[0]`, por ejemplo, para acceder al primer elemento, `tableRowsEven[1]` para acceder al segundo elemento, y así sucesivamente. Otra característica común es la propiedad `length`, que se puede utilizar para determinar el número de elementos de la colección.

Así, por ejemplo, para iterar sobre todos los elementos de una colección, se puede proceder como se muestra en **el Listado 1.7**. Este listado utiliza un bucle `for` para iterar sobre todos los elementos de la colección. Al igual que con la iteración sobre matrices reales, se puede utilizar la propiedad `length` y el acceso por índice. Utilizando estas características, el ejemplo asigna un nuevo color de fondo a cada elemento de la colección (véase **la Figura 1.7**).

```
const tableRowsEven = document
    .getElementsByClassName('even'); // Selecciona todas las filas pares de la
    tabla
if(tableRowsEven.length >0) { // Si se encuentra al menos un elemento
    for(let i=0; i<tableRowsEven.length; i++) { // Iterar todos los
    elementos.
        const tableRow = tableRowsEven[i]; // Asignar elemento a una variable.
        tableRow.style.backgroundColor = '#CCCCC'; // Establecer nuevo color de
    fondo.
    }
}
```

Listado 1.7 Iteración sobre una colección utilizando la sintaxis de matriz

Contacts		
First name	Last name	E-mail address
John	Doe	john.doe@javascripthandbuch.de
James	Doe	james.doe@javascripthandbuch.de
Peter	Doe	peter.doe@javascripthandbuch.de
Paul	Doe	paul.doe@javascripthandbuch.de

Figura 1.7 A las celdas pares de la tabla se les asignó un color de fondo diferente mediante JavaScript

Modificación del CSS de un elemento

La propiedad `style` de un elemento permite acceder o modificar las propiedades CSS de un elemento. El objeto almacenado en esta propiedad contiene todas las propiedades CSS como propiedades de objeto (`style.color`, `style.border`, etc.). Para las propiedades CSS que contienen un guión, como `background-color`, las propiedades de objeto correspondientes se definen en notación CamelCase (por ejemplo, `style.backgroundColor` o `style.fontFamily`).

Como alternativa a la sintaxis de matriz con corchetes, se puede acceder a los elementos individuales de la colección utilizando el método `item()`. En este caso, el argumento que se debe pasar es el índice del elemento que se va a devolver. El bucle anterior también se podría reformular como se muestra en **el Listado 1.8**.

```
const tableRowsEven = document
  .getElementsByClassName('even'); // Selecciona todas las filas pares de la
  tabla
if(tableRowsEven.length > 0) { // Si se encuentra al menos un elemento
  for(let i=0; i<tableRowsEven.length; i++) { // Iterar todos los elementos.
    const tableRow = tableRowsEven.item(i); // Asignar el elemento a una
    variable.
    tableRow.style.backgroundColor = '#CCCCCC'; // Establecer nuevo color de
    fondo.
  }
}
```

Listado 1.8 Iteración sobre una colección utilizando el método `item()`

Préstamo de métodos

Dado que estas colecciones no son matrices reales (son objetos del tipo `HTMLCollection` o `NodeList`), sino objetos similares a matrices, en la práctica a menudo utilizarás la técnica de *préstamo de métodos* para poder seguir utilizando métodos de matriz.

```
Array.prototype.forEach.call(tableRowsEven, (tableRow) => {
  tableRow.style.backgroundColor = '#CCCCCC';
});
```

Listado 1.9 Iteración sobre una colección mediante el préstamo de métodos

En ES6 existen alternativas más legibles que el préstamo de métodos: recorrer la colección con un bucle `for...of` o convertirla en una matriz real con `Array.from()` o con el operador de propagación (*spread*). Además, los objetos `NodeList` (por ejemplo, el que devuelve `querySelectorAll()`) ya incluyen el método `forEach()`.

```
// 1. Bucle for...of
for (const tableRow of tableRowsEven) {
  tableRow.style.backgroundColor = '#CCCCCC';
}

// 2. Convertir en matriz con Array.from()
Array.from(tableRowsEven).forEach((tableRow) => {
```

```
    tableRow.style.backgroundColor = '#CCCCCC';
  });

// 3. Convertir en matriz con el operador spread
[...tableRowsEven].forEach((tableRow) => {
  tableRow.style.backgroundColor = '#CCCCCC';
});
```

Listado 1.9b Alternativas ES6 para recorrer una colección

Colecciones activas (vivas) frente a colecciones estáticas

Se distingue entre colecciones activas (vivas) y estáticas. Una colección activa está conectada al DOM: si después de obtenerla se añaden o se eliminan elementos que cumplen el criterio de búsqueda, la colección se actualiza automáticamente (por ejemplo, cambia su propiedad `length` sin volver a llamar al método).

Una colección estática, en cambio, es una «foto fija» del DOM en el momento de la consulta: no refleja los elementos que se añadan o eliminen después. En ambos casos, los elementos que contiene son los reales de la página, así que si modificas uno (por ejemplo, su estilo) el cambio sí se ve inmediatamente; la diferencia está solo en si la colección «se entera» de los elementos añadidos o eliminados. Los métodos `getElementsByClassName()`, `getElementsByTagName()` y `getElementsByName()` devuelven colecciones activas, mientras que el método `querySelectorAll()` devuelve una lista de nodos (`NodeList`) estática.

Veamos la diferencia con un ejemplo sobre la página del **Listado 1.20** (listas anidadas con 8 entradas). Tras añadir una nueva entrada, la colección activa se actualiza sola, pero la estática no:

```
const vivas = document.getElementsByTagName('li');
const estaticas = document.querySelectorAll('li');
console.log(vivas.length, estaticas.length);    // 8 8

// Añadimos un nuevo <li> (createElement y append se verán en la UT 3.2)
document.querySelector('ul').append(document.createElement('li'));
console.log(vivas.length, estaticas.length);    // 9 8
```

Listado 1.9c Diferencia entre una colección activa y una estática

1.2.3 Selección de elementos por nombre de elemento

El método `getElementsByTagName()` se puede utilizar para seleccionar elementos por su nombre. El método espera el nombre del elemento. Por ejemplo, para seleccionar todas las celdas de la tabla (véase **la figura 1.8**), proceda como se muestra en **el listado 1.10**.

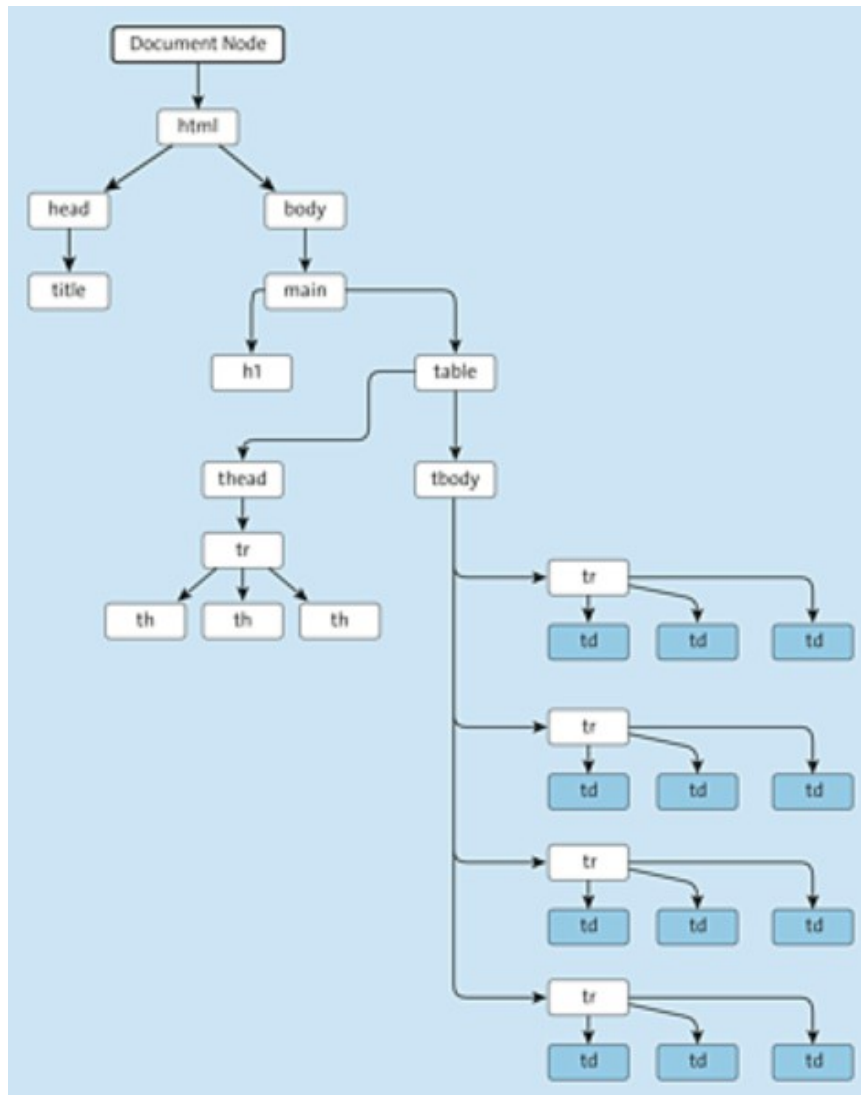


Figura 1.8 El método `getElementsByName()` selecciona elementos por su nombre

En primer lugar, se seleccionan todas las celdas de la tabla mediante el método `getElementsByName()` y, a continuación, se asigna a cada elemento una nueva fuente y un nuevo tamaño de fuente. Los resultados se muestran en **la figura 1.9**.

```
const tableCells = document.getElementsByTagName('td');
if(tableCells.length > 0) { // Si se encuentra al menos un elemento

    for(let i=0; i<tableCells.length; i++) { // Iterar todos los elementos.
        const tableCell = tableCells[i]; // Asignar el elemento a una variable.
        tableCell.style.fontFamily = 'Verdana'; // Establecer nueva fuente.
        tableCell.style.fontSize = '9pt'; // Establecer nuevo tamaño de fuente.
    }
}
```

Listado 1.10 Acceso a un elemento mediante el nombre del elemento

Contacts		
First name	Last name	E-mail address
John	Doe	john.doe@javascripthandbuch.de
James	Doe	james.doe@javascripthandbuch.de
Peter	Doe	peter.doe@javascripthandbuch.de
Paul	Doe	paul.doe@javascripthandbuch.de

Figura 1.9 Se asigna una nueva fuente y un nuevo tamaño de fuente a las celdas de la tabla

Nota

Asegúrese de pasar solo el nombre del elemento al método `getElementsByTagName()`, sin corchetes angulares adicionales ni similares. Por ejemplo, la llamada `getElementsByTagName(' <td>')` no funcionaría.

1.2.4 Selección de elementos por nombre

A algunos elementos se les puede asignar un atributo de nombre en HTML, por ejemplo, los elementos `<input>` de tipo radio, para indicar su asociación con un grupo de selección. En el **Listado 1.11**, por ejemplo, los tres botones de radio están asignados al grupo género.

```
<form action="">
  <label for="artist">Artist</label>
  <input id="artist" type="text" name="artist">
  <br>
  <label for="album">Album</label>
  <input id="album" type="text" name="album">
  <br>
  <p>Genre:</p>
  <fieldset>
    <input type="radio" id="st" name="genre" value="Stonerrock">
    <label for="st">Stonerrock</label>
    <br>
    <input type="radio" id="sp" name="genre" value="Spacerock">
    <label for="sp">Spacerock</label>
    <br>
    <input type="radio" id="ha" name="genre" value="Hardrock">
    <label for="ha">Hardrock</label>
  </fieldset>
</form>
```

Listado 1.11 Un formulario HTML sencillo

Con el método `getElementByName()`, se pueden seleccionar elementos basándose en este atributo de `name`. En el **Listado 1.12**, este método se utiliza para seleccionar todos los elementos con un atributo de `name` con el valor **genre**. (Sin embargo, los otros dos elementos del formulario con los valores `artist` y `album` no se seleccionan; véase la **Figura 1.10**).

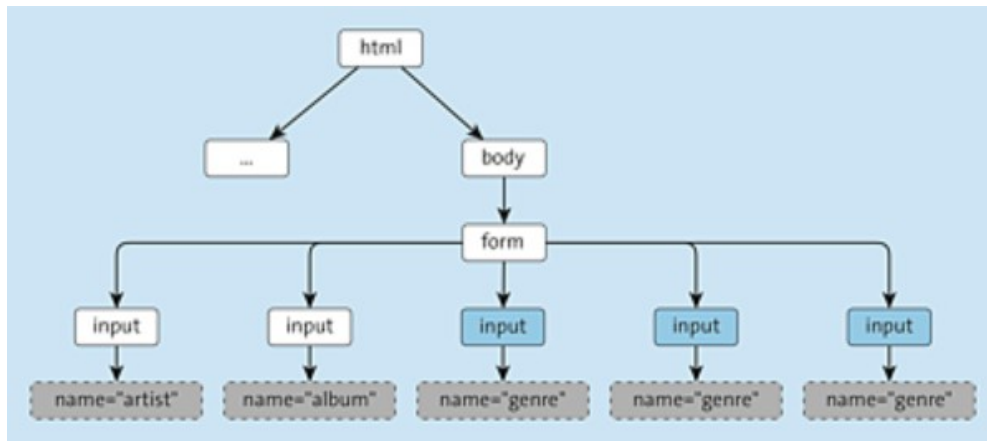


Figura 1.10 El método `getElementsByName()` selecciona elementos por su atributo `name`

En el bucle del **Listado 1.12**, se muestran los valores de estos elementos (`inputElement.value`): Stonerrock, Spacerock y Hardrock.

```

const inputElementsForGenre = document
    .getElementsByName('genre'); // Selecciona todos los elementos por nombre.
if(inputElementsForGenre.length > 0) { // Si se encuentra al menos un
    elemento.
    for(let i=0; i<inputElementsForGenre.length; i++) { // Iterar todos los
        elementos.
        const inputElement = inputElementsForGenre[i]; // Asignar elemento a una
                                                    // variable.
        console.log(inputElement.value); // Salida: Stonerrock,
                                                    // Spacerock, Hardrock
    }
}

```

Listado 1.12 Acceso a elementos por sus nombres

Compatibilidad de los navegadores con `getElementsByName()`

El método `getElementsByName()` no funciona de manera consistente en todos los navegadores. Por ejemplo, en algunas versiones de los navegadores Internet Explorer y Opera, el método devuelve no solo aquellos elementos con un atributo `name` que coincide con el valor pasado, sino también aquellos con un atributo `id` que coincide con el valor pasado. En mi opinión, los otros métodos de selección (presentados hasta ahora y que se presentarán en breve) ofrecen posibilidades suficientes para seleccionar elementos, de modo que, en la práctica, se puede prescindir de este método.

1.2.5 Selección de elementos por selector

Los métodos DOM para seleccionar elementos presentados hasta ahora ya logran mucho, pero son bastante restringidos en cuanto a la forma de expresión. Es posible que el elemento que se desea seleccionar no tenga ningún ID o clase; en tales casos, los métodos `getElementById()` o `getElementsByClassName()` no sirven de ayuda. Por otro lado, el método `getElementsByTagName()` es muy poco específico, ya que tiende a seleccionar muchos elementos.

Y `getElementsByName()` debe utilizarse con precaución, por las razones mencionadas anteriormente.

Los métodos `querySelector()` y `querySelectorAll()` son mucho más versátiles y expresivos para devolver elementos para un selector CSS determinado. El primer método devuelve el *primer elemento* que coincide con el selector CSS correspondiente, mientras que el segundo devuelve *todos los elementos* que coinciden con el selector CSS.

El Listado 1.13 muestra un ejemplo del uso de `querySelector()`. Este ejemplo pasa el selector CSS `#main table td:nth-child(2)`, que describe las segundas celdas de cada fila (`td:nth-child(2)`) dentro de una tabla (`table`) dentro de un elemento con el ID principal (`#main`). Pero como el método `querySelector()` solo selecciona el primer elemento que coincide con un selector, solo se devuelve el primer elemento `<td>` (véanse **la figura 1.11** y **la figura 1.12**).

```
const tableCell = document.querySelector('#main table td:nth-child(2)');
tableCell.style.border = 'thick solid red';
```

Listado 1.13 Acceso a un elemento mediante el selector CSS

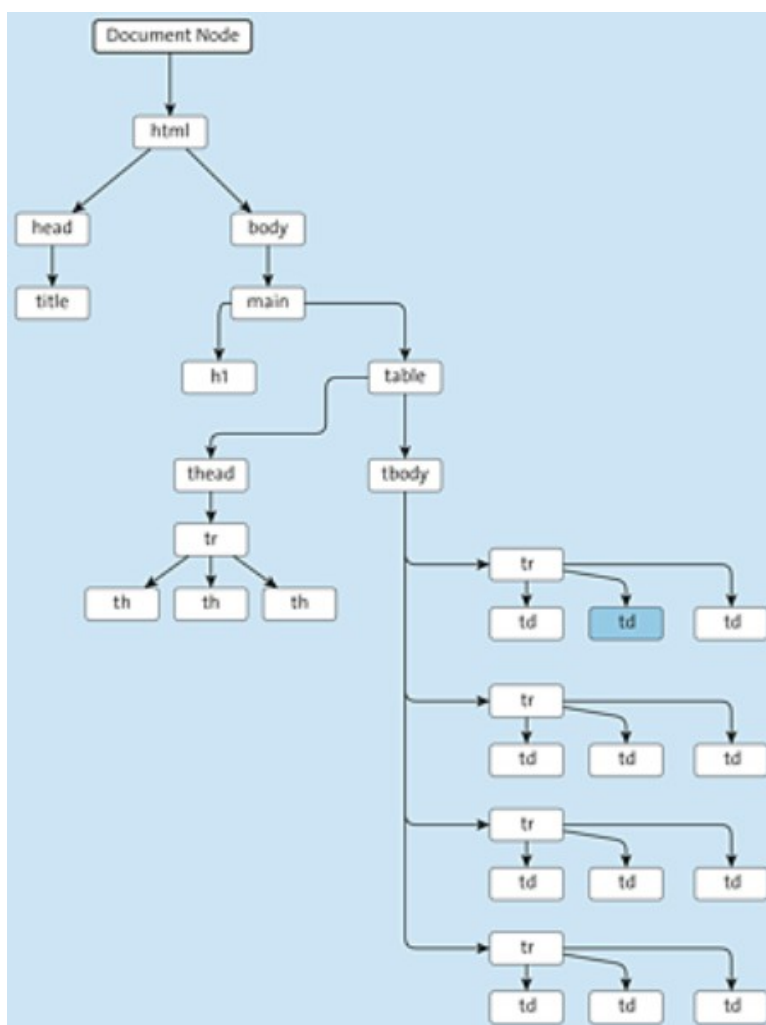


Figura 1.11 El método `querySelector()` devuelve como máximo un elemento

Contacts

First name	Last name	E-mail address
John	Doe	john.doe@javascripthandbuch.de
James	Doe	james.doe@javascripthandbuch.de
Peter	Doe	peter.doe@javascripthandbuch.de
Paul	Doe	paul.doe@javascripthandbuch.de

Figura 1.12 El método `querySelector()` devuelve el primer elemento que coincide con el selector CSS

Por otro lado, el **Listado 1.14** muestra el uso del método `querySelectorAll()`. Este listado utiliza el mismo selector CSS que el anterior. Sin embargo, en esta ocasión se obtienen *todos* los elementos que coinciden con este selector, es decir, todos los segundos elementos `<td>` (véase **la Figura 1.13**).

```
const tableCells = document.querySelectorAll('#main table td:nth-child(2)');
if(tableCells.length > 0) {
  for(let i=0; i<tableCells.length; i++) {
    const tableCell = tableCells[i];
    tableCell.style.border = 'thick solid red';
  }
}
```

Listado 1.14 Acceso a varios elementos mediante el selector CSS

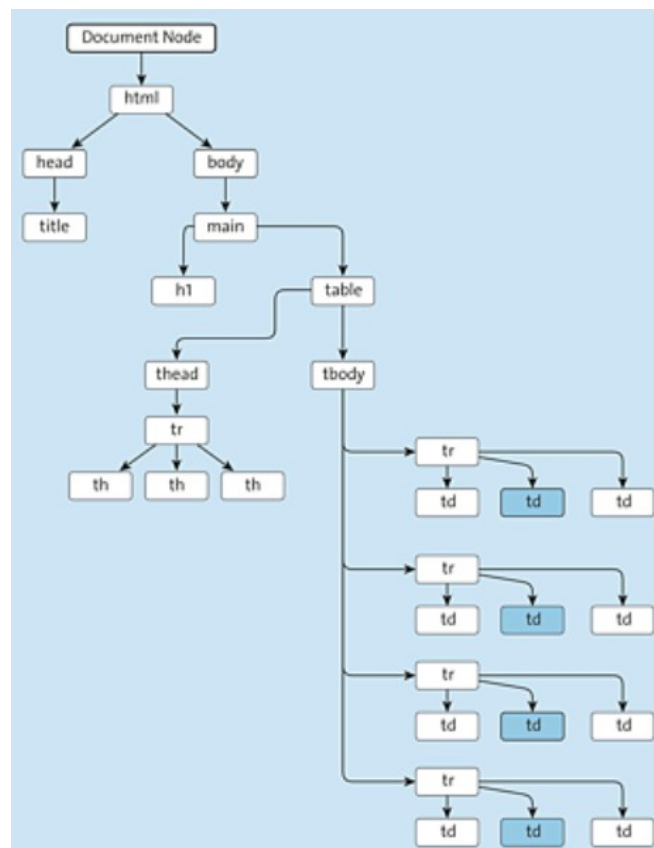


Figura 1.13 El método `querySelectorAll()` puede devolver varios elementos

Dentro del bucle, estos elementos se marcan con un marco rojo de la misma manera que antes (véase **la figura 1.14**).

Contacts		
First name	Last name	E-mail address
John	Doe	john.doe@javascripthandbuch.de
James	Doe	james.doe@javascripthandbuch.de
Peter	Doe	peter.doe@javascripthandbuch.de
Paul	Doe	paul.doe@javascripthandbuch.de

Figura 1.14 El método `querySelectorAll()` devuelve todos los elementos que coinciden con el selector CSS pasado

En general, los métodos para seleccionar elementos mediante selectores CSS facilitan mucho el trabajo de los desarrolladores de JavaScript. En **la tabla 1.3** se muestra una descripción general de los diferentes selectores CSS.

Selector	Descripción
*	Selecciona todos los elementos
E	Selecciona elementos de tipo E
[a]	Selecciona elementos con el atributo a
[a="b"]	Selecciona elementos con el atributo a que tiene el valor b
[a~="b"]	Selecciona elementos con el atributo a cuyo valor es una lista de valores, uno de los cuales es igual a b
[a^="b"]	Selecciona elementos con el atributo a cuyo valor comienza por b.
[a\$="b"]	Selecciona elementos con el atributo a cuyo valor termina con b.
[a*="b"]	Selecciona elementos con el atributo a cuyo valor contiene b como subcadena.
[a "b"]	Selecciona elementos con el atributo a cuyos valores son una serie de valores separados por signos menos, donde el primer valor es b
:root	Selecciona el elemento raíz de un documento
:nth-child(n)	Selecciona el elemento hijo n-ésimo de un elemento
:nth-last-child(n)	Selecciona el elemento hijo n-ésimo de un elemento empezando

	por el final
:nth-of-type(n)	Selecciona el elemento hermano n-ésimo de un tipo específico de un elemento
:nth-last-of-type(n)	Selecciona el elemento hermano n-ésimo de un tipo específico de un elemento, comenzando desde el final.
:first-child	Selecciona el primer elemento hijo de un elemento.
:last-child	Selecciona el último elemento secundario de un elemento.
:first-of-type	Selecciona el primer elemento hermano de un elemento
:last-of-type	Selecciona el último elemento hermano de un elemento.
:only-child	Selecciona los elementos que son el único elemento hijo de su elemento padre
:only-of-type	Selecciona los elementos que son el único elemento de su tipo entre sus elementos hermanos.
:empty	Selecciona elementos que no tienen ningún hijo (ni elementos ni texto).
:link	Selecciona enlaces en los que aún no se ha hecho clic
:visited	Selecciona los enlaces en los que ya se ha hecho clic
:active	Selecciona los enlaces en los que se está haciendo clic en ese momento
:hover	Selecciona los enlaces sobre los que se encuentra actualmente el ratón.
:focus	Selecciona los enlaces que están actualmente enfocados
:target	Selecciona las etiquetas de salto a las que se puede llegar a través de enlaces dentro de una página web
:lang(en)	Selecciona elementos cuyo atributo lang tiene el valor en
:enabled	Selecciona elementos de formulario en los que se pueden introducir valores o que se pueden manejar (y que no están desactivados).
:disabled	Selecciona los elementos del formulario que no se pueden utilizar o cuya entrada se ha desactivado mediante el atributo disabled.
:checked	Selecciona casillas de verificación y botones de opción que

	están marcados (seleccionados).
.className	Selecciona los elementos cuyo atributo de clase tiene el valor className
#main	Selecciona los elementos cuyo atributo id tiene el valor main
:not(s)	Selecciona elementos que no coinciden con el selector s especificado entre paréntesis
E F	Selecciona elementos de tipo F que aparecen en algún lugar dentro de un elemento de tipo E
E > F	Selecciona elementos de tipo F que son elementos secundarios de un elemento de tipo E
E + F	Selecciona elementos de tipo F que son elementos hermanos inmediatamente posteriores a un elemento de tipo E.
E ~ F	Selecciona elementos de tipo F que son elementos hermanos de un elemento de tipo E.

Tabla 1.3 Los diferentes selectores en CSS3

1.2.6 Selección del elemento padre de un elemento

Los nodos de elemento tienen varias propiedades que permiten acceder a elementos relacionados. Los elementos relacionados son nodos o elementos padres, nodos o elementos hijos y nodos o elementos hermanos.

Para la selección de nodos/elementos padres, se dispone de las propiedades parentNode y parentElement; para la selección de nodos/elementos hijos, existen las propiedades firstChild, firstElementChild, lastChild, lastElementChild, childNodes y children; y para la selección de nodos/elementos hermanos, se proporcionan las propiedades previousSibling, previousElementSibling, nextSibling y nextElementSibling.

Estas características se explicarán con más detalle a continuación. Comencemos con la selección de nodos o elementos padres.

Para seleccionar el nodo padre de un elemento (o nodo), se puede utilizar la propiedad parentNode, mientras que el elemento padre se puede seleccionar mediante parentElement. En la mayoría de los casos, el nodo padre también es un elemento, es decir, las propiedades parentNode y parentElement contienen el mismo valor (véase el **Listado 1.15** y la **Figura 1.15**).

```
const table = document.querySelector('table');
console.log(table.parentNode); // <main>
console.log(table.parentElement); // <main>
```

Listado 1.15 Acceso a un nodo padre o elemento padre

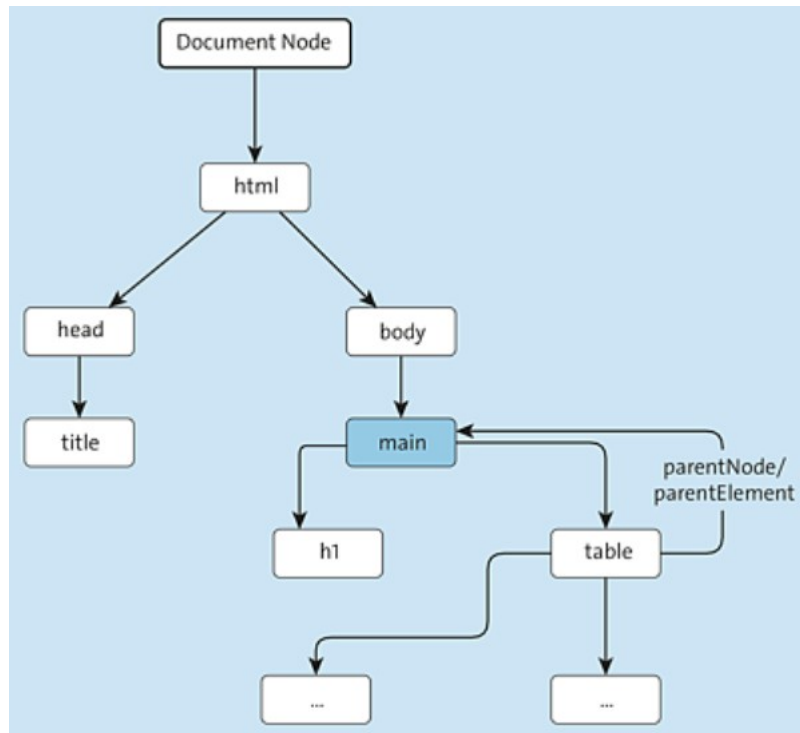


Figura 1.15 Selección del elemento padre

Nodos y elementos

No todos los nodos del árbol DOM son elementos, pero todos los elementos son siempre nodos.

Es importante comprender que algunas de las propiedades anteriores devuelven nodos, mientras que otras devuelven elementos. Las propiedades `parentNode`, `childNodes`, `firstChild`, `lastChild`, `previousSibling` y `nextSibling` devuelven nodos, mientras que las propiedades `parentElement`, `children`, `firstElementChild`, `lastElementChild`, `previousElementSibling` y `nextElementSibling` devuelven elementos.

El siguiente ejemplo ilustra lo que esto significa en la práctica. Observe el código HTML del **Listado 1.16** y su DOM en la **Figura 1.16**. Muestran una página web relativamente sencilla con solo dos elementos `<span>` dentro del elemento `<body>`, cada uno de ellos encerrado en texto.

```

<!DOCTYPE html>
<html>
<body>
  Texto
  <span></span>
  Texto
  <span></span>
  Texto
</body>
</html>

```

Listado 1.16 Ejemplo sencillo de HTML

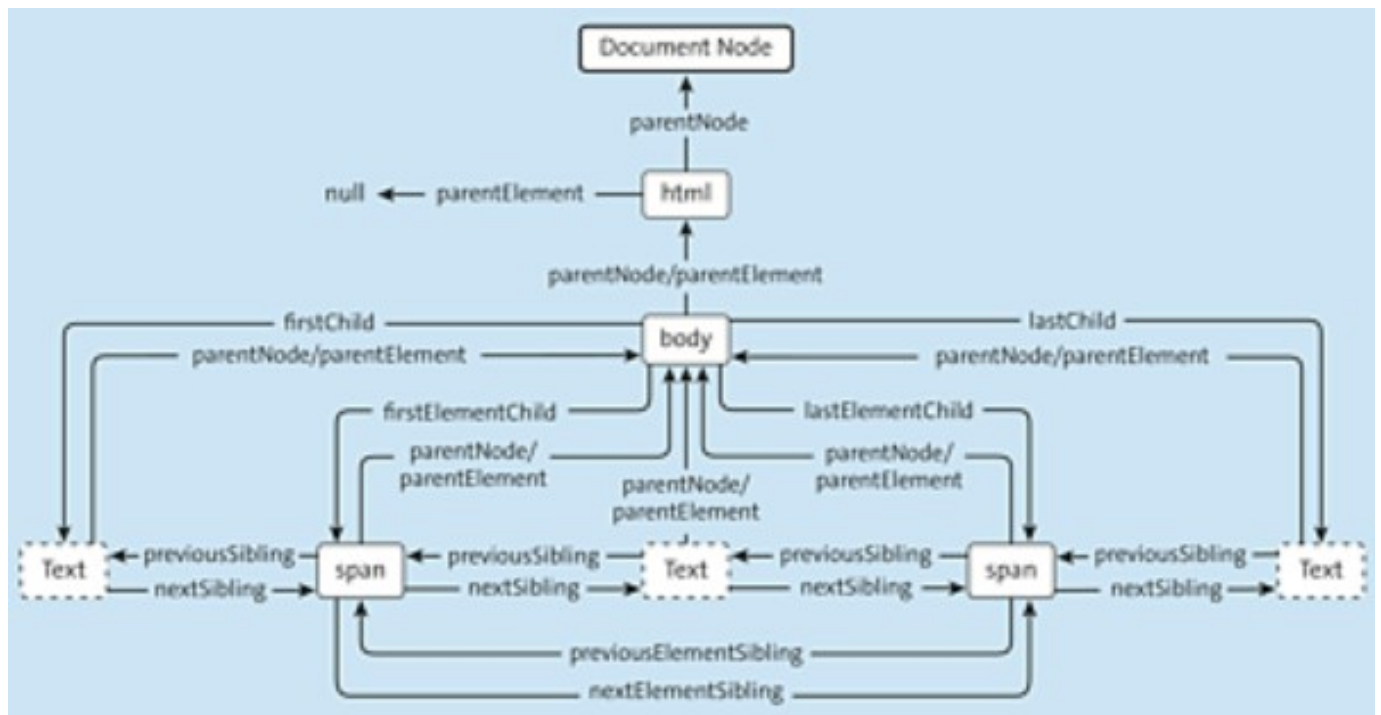


Figura 1.16 Descripción general de las diferentes formas de acceso

En consecuencia, el DOM correspondiente debajo del elemento `<body>` contiene (en este orden) un nodo de texto, un nodo de elemento, un nodo de texto, un nodo de elemento y, de nuevo, un nodo de texto. Para todos estos nodos, el elemento `<body>` representa al mismo tiempo el nodo padre y el elemento padre. Por lo tanto, para todos estos nodos, las propiedades `parentNode` y `parentElement` devuelven el mismo valor: el elemento `<body>`.

Además, en el DOM de **la figura 1.16** se puede ver que las propiedades `parentNode` y `parentElement` suelen hacer referencia siempre al mismo elemento para todos los nodos. La única excepción es el elemento `<html>`. Este elemento no tiene un elemento padre, sino «solo» un nodo padre: el nodo del documento. En este caso, la propiedad `parentElement` devuelve el valor `null`.

Las demás relaciones entre elementos y nodos en el DOM se tratarán en las siguientes secciones.

1.2.7 Selección de los elementos secundarios de un elemento

Los elementos secundarios de un elemento se pueden determinar mediante la propiedad `children`, y los nodos secundarios mediante la propiedad `childNodes`. Se puede determinar si un elemento tiene nodos secundarios utilizando el método `hasChildNodes()`, que devuelve un valor booleano. Se puede determinar si un elemento tiene elementos secundarios utilizando la propiedad `childElementCount`: esta contiene el número de elementos secundarios.

El Listado 1.17 muestra algunos ejemplos (de nuevo relacionados con el HTML del **Listado 1.1**). Se puede ver que el elemento `<tbody>` tiene cuatro elementos secundarios (los cuatro elementos `<tr>` Véase **la Figura 1.17**) y un total de nueve nodos secundarios (véase **la Figura 1.18**).

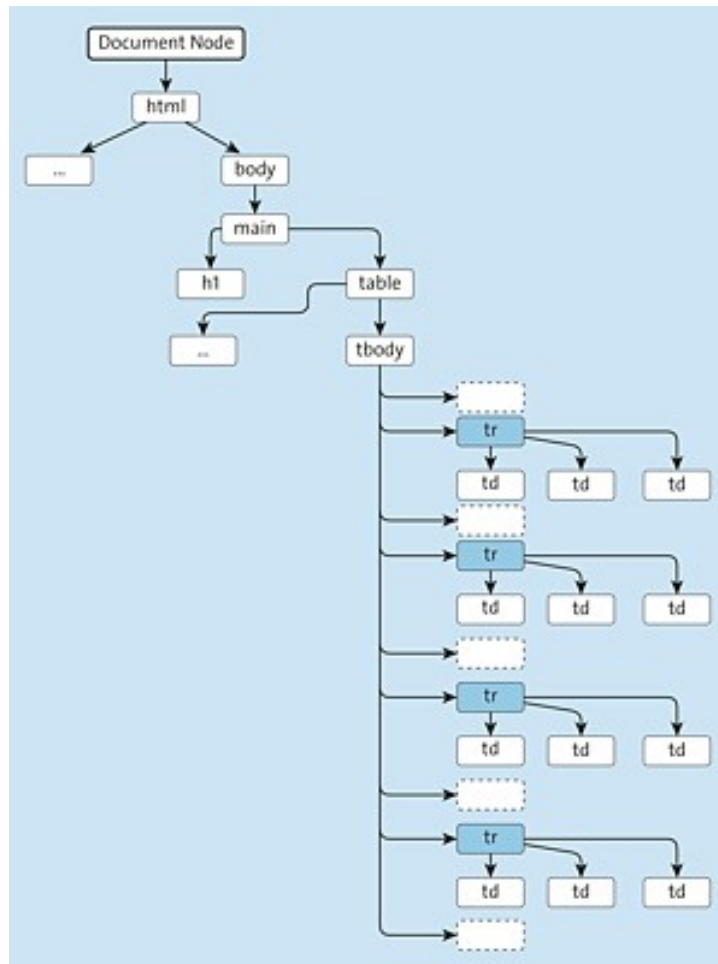


Figura 1.17 Selección de todos los elementos secundarios

```
const tbody = document.querySelector('tbody');
console.log(tbody.children.length); // 4
console.log(tbody.childElementCount); // 4
console.log(tbody.childNodes.length); // 9
console.log(tbody.hasChildNodes()); // true
```

Listado 1.17 Acceso a nodos hijos o elementos hijos

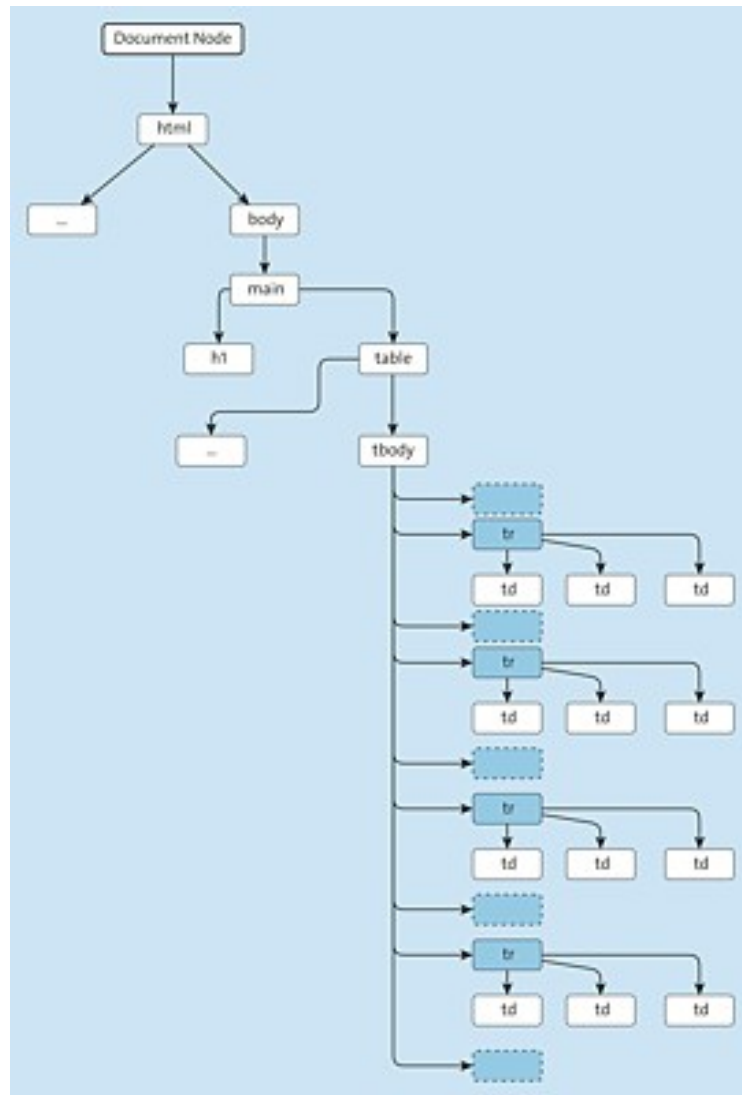


Figura 1.18 Selección de todos los nodos secundarios

La razón de esto es que, aunque no hay texto entre, antes y después de los cuatro elementos `<tr>` en HTML, se producen los llamados nodos de espacio en blanco (véase el siguiente recuadro). Estos nodos de espacio en blanco se crean siempre que se utilizan saltos de línea entre dos elementos en HTML, por ejemplo.

Nodos de espacio en blanco

Los espacios en blanco dentro del código HTML, creados por espacios, tabulaciones o saltos de línea, por ejemplo, siempre dan lugar a nodos de texto sin texto en el DOM. Se denominan *nodos de espacio en blanco*.

Además, hay otras propiedades disponibles que se pueden utilizar para seleccionar elementos secundarios o nodos secundarios individuales:

La propiedad `firstChild` contiene el primer nodo secundario.

La propiedad `lastChild` contiene el último nodo secundario.

La propiedad `firstElementChild` contiene el primer elemento secundario.

La propiedad `lastElementChild` contiene el último elemento secundario.

El **Listado 1.18** muestra algunos ejemplos. El resultado de seleccionar los primeros y últimos elementos secundarios se muestra en la **Figura 1.19**, y el resultado de seleccionar los primeros y últimos nodos secundarios se ilustra en la **Figura 1.20**.

```
const tbody = document.querySelector('tbody');
console.log(tbody.firstChild);           // Nodo de texto
console.log(tbody.lastChild);            // Nodo de texto
console.log(tbody.firstChildElementChild); // <tr>
console.log(tbody.lastElementChild);     // <tr>
```

Listado 1.18 Acceso a nodos hijos y elementos hijos específicos

Nota

En la mayoría de los casos, probablemente trabajarás con nodos de elementos. En estos casos, lo mejor es utilizar propiedades que también devuelvan nodos de elementos (como `firstElementChild` y `lastElementChild`). Hubo un tiempo en que los desarrolladores web solo disponían de propiedades que devolvían todo tipo de nodos (por ejemplo, `firstChild` y `lastChild`) y tenían que filtrar los nodos de elementos en función del tipo de nodo en sí. Afortunadamente, esto ha cambiado.

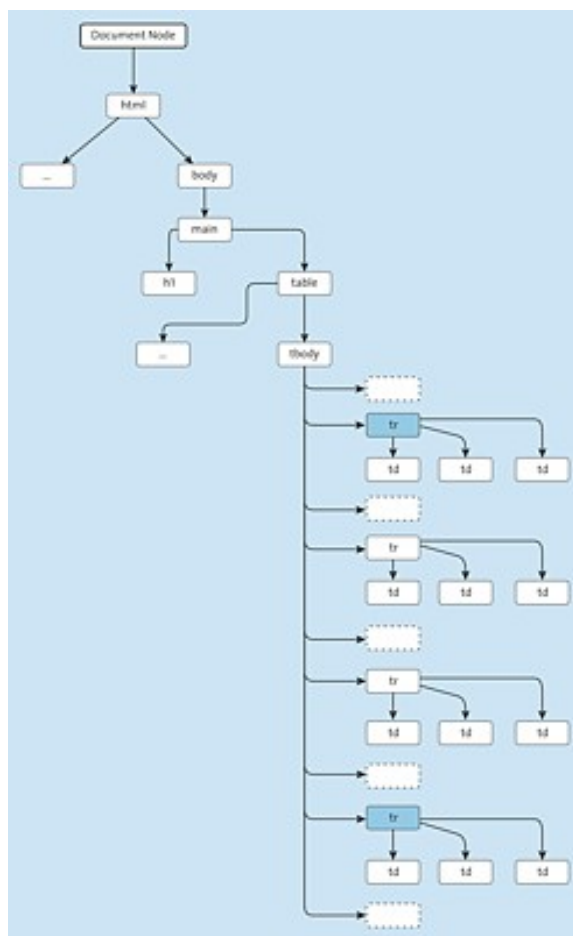


Figura 1.19 Selección del primer y último elemento secundario

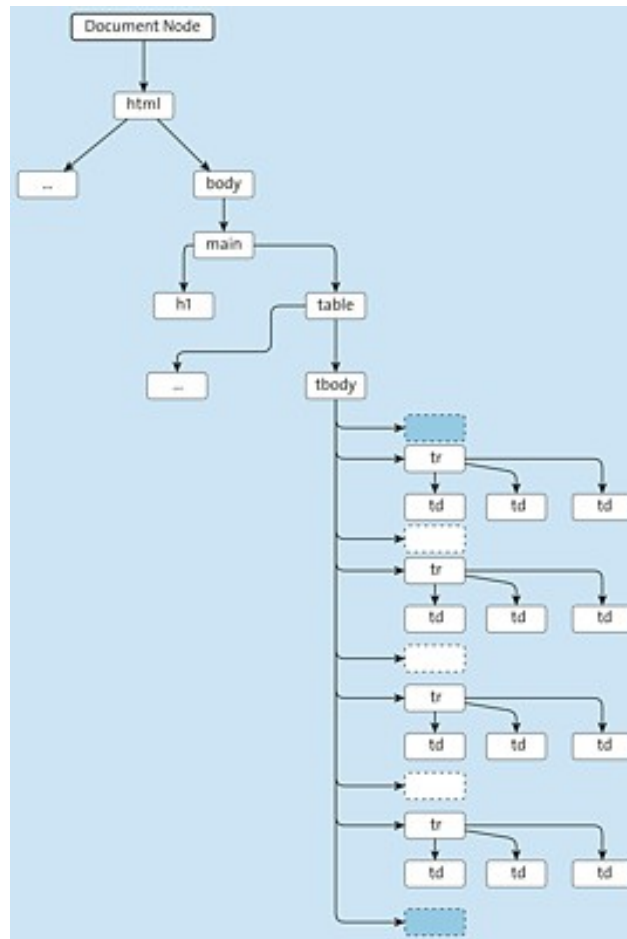


Figura 1.20 Selección del primer y último nodo hijo

1.2.8 Selección de los elementos hermanos de un elemento

Ahora ya sabes cómo seleccionar nodos/elementos en el árbol DOM por encima (nodos/elementos padres) y por debajo (nodos/elementos hijos) de un nodo/elemento inicial utilizando sus propiedades. Además, también existe la posibilidad de seleccionar los nodos hermanos o elementos hermanos *dentro de un nivel* del DOM:

La propiedad `previousSibling` contiene el nodo hermano anterior.

La propiedad `nextSibling` contiene el siguiente nodo hermano.

La propiedad `previousElementSibling` contiene el elemento hermano anterior.

La propiedad `nextElementSibling` contiene el siguiente elemento hermano.

En el **Listado 1.19** se muestra un ejemplo de código. Comenzando por la segunda fila de la tabla, primero se seleccionan el nodo hermano anterior (mediante `previousSibling`) y el nodo hermano siguiente (mediante `nextSibling`), ambos nodos de texto o, más precisamente, nodos de espacio en blanco (véase la **Figura 1.21**).

```
const tableCell = document.querySelector('tbody tr:nth-child(2)');
console.log(tableCell.previousSibling);           // Nodo de texto
console.log(tableCell.nextSibling);               // Nodo de texto
console.log(tableCell.previousElementSibling);    // <tr>
console.log(tableCell.nextElementSibling);        // <tr>
```

Listado 1.19 Acceso a nodos hermanos específicos y elementos hermanos

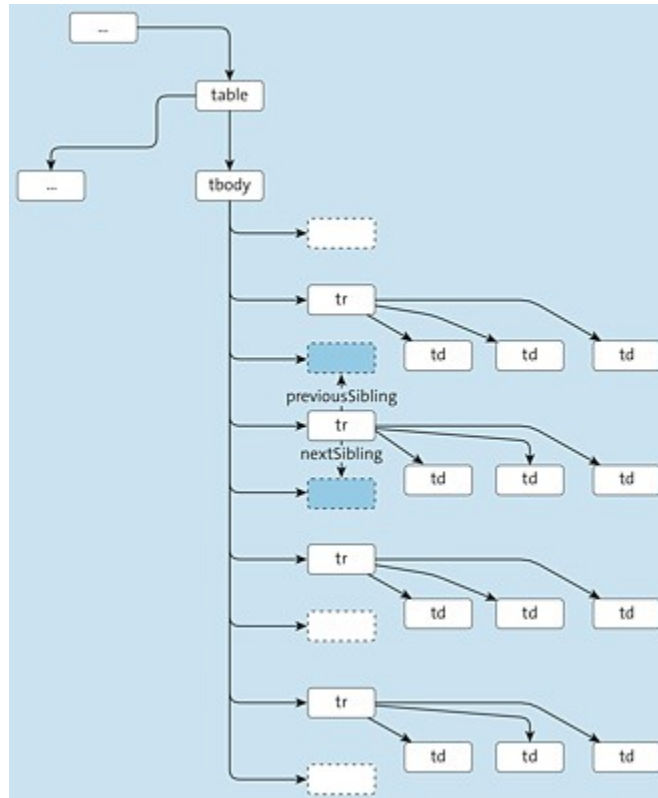


Figura 1.21 Selección del nodo hermano anterior y posterior

A continuación, `previousElementSibling` selecciona el elemento hermano anterior y `nextElementSibling` selecciona el elemento hermano siguiente (véase **la figura 1.22**).

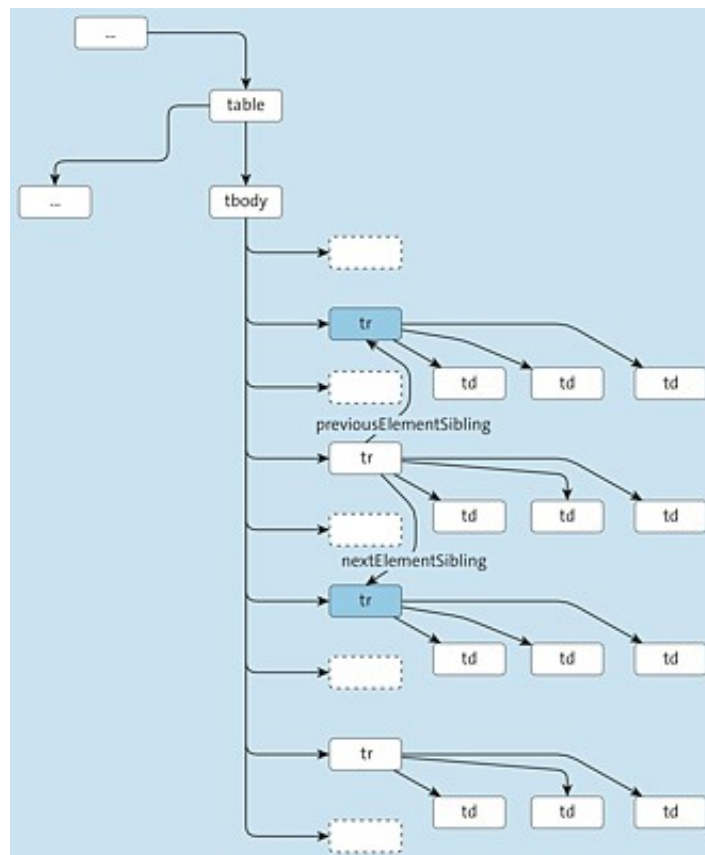


Figura 1.22 Selección del elemento hermano anterior y posterior

1.2.9 Llamada a métodos de selección en elementos

La mayoría de los métodos DOM presentados para seleccionar elementos (`getElementsByClassName()`, `getElementsByTagName()`, `querySelector()` y `querySelectorAll()`) se pueden invocar no solo en el nodo del documento (es decir, en `document`), sino también en todos los demás nodos de elementos de una página web (solo `getElementById()` y `getElementsByName()` se pueden invocar únicamente en el nodo del documento). En este caso, la búsqueda de elementos solo incluye el subárbol situado debajo del elemento en el que se ha invocado el método correspondiente.

Consideremos el código HTML del **Listado 1.20**, que contiene listas anidadas. En el código JavaScript que se muestra en el **Listado 1.21**, el método `getElementsByTagName()` con el argumento `li` se invoca primero en el nodo `document` (que selecciona todas las entradas de la lista de toda la página web; véase **la figura 1.23**) y, a continuación, en la lista anidada con el ID `list-2` (que, a su vez, selecciona solo aquellas entradas de la lista que aparecen en este subárbol del DOM, es decir, debajo de la lista anidada; véase **la figura 1.24**).

```
<!DOCTYPE html>
<html>
  <head lang="en">
    <title>Ejemplo de selección de elementos</title>
  </head>
  <body>
    <main id="main-content">
      <ul id="list-1">
        <li>Entrada de lista 1</li>
        <li>
          Entrada de lista 2
          <ul id="list-2">
            <li>Entrada de lista 2.1</li>
            <li>Entrada de lista 2.2</li>
            <li>Entrada de lista 2.3</li>
            <li>Entrada de lista 2.4</li>
          </ul>
        </li>
        <li>Entrada de lista 3</li>
        <li>Entrada de lista 4</li>
      </ul>
    </main>
  </body>
</html>
```

Listado 1.20 Ejemplo de página HTML

```
const allListItemElements = document.getElementsByTagName('li');
console.log(allListItemElements.length); // Salida: 8
const subList = document.getElementById('list-2');
```



```
const subListListItems = subList.getElementsByTagName('li');  
console.log(subListListItems.length); // Salida: 4
```

Listado 1.21 Selección de elementos a partir de un elemento padre

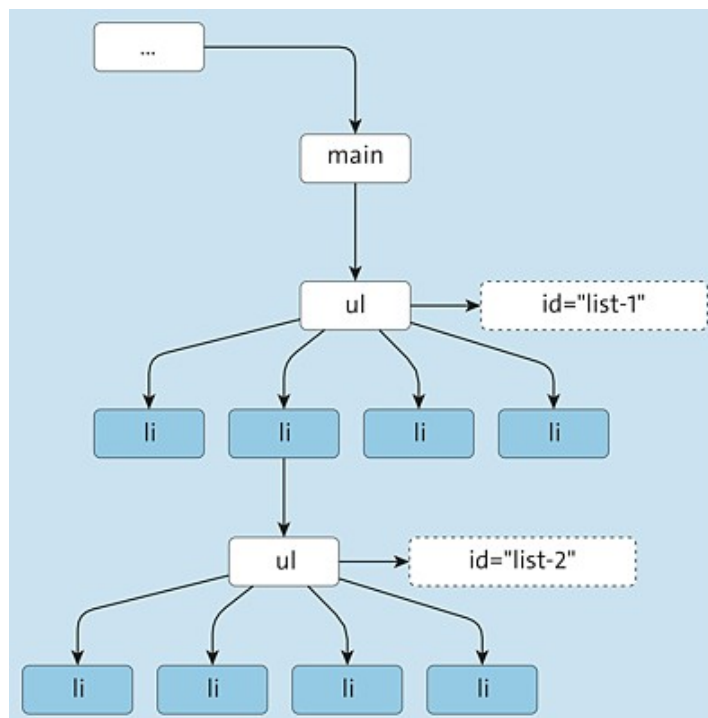


Figura 1.23 Llamada al método `getElementsByTagName()` en el nodo del documento

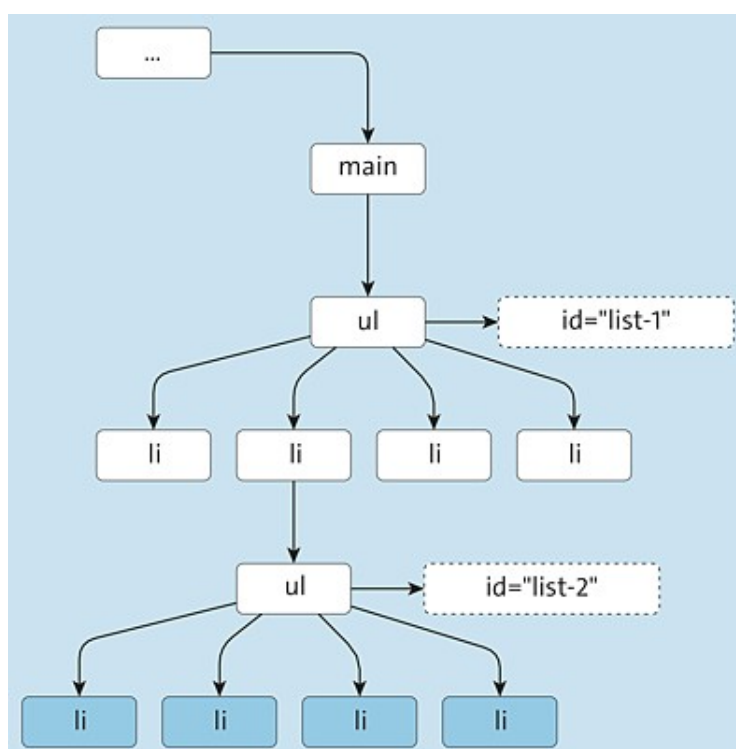


Figura 1.24 Llamada al método `getElementsByTagName()` en el elemento `<ul>` con el ID `list-2`

1.2.10 Selección de elementos por tipo

Además de los métodos de selección presentados, el objeto `document` proporciona varias propiedades para acceder directamente a elementos específicos de una página web, véase la **Tabla 1.4**. Por ejemplo, la propiedad `anchors` se puede utilizar para seleccionar todos los

anclajes (es decir, elementos de salto) de una página web, `forms` para seleccionar todos los formularios, `images` para seleccionar todas las imágenes y `links` para seleccionar todos los enlaces. Además, la propiedad `head` está disponible para acceder directamente al elemento `<head>`, y la propiedad `body` se puede utilizar para acceder directamente al elemento `<body>`.

Propiedad	Descripción
<code>document.anchors</code>	Contiene una lista de todos los anclajes de la página web
<code>document.forms</code>	Contiene una lista de todos los formularios de la página web
<code>document.images</code>	Contiene una lista de todas las imágenes de la página web
<code>document.links</code>	Contiene una lista de todos los enlaces de la página web
<code>document.head</code>	Proporciona acceso al elemento <code><head></code> de la página web
<code>document.body</code>	Proporciona acceso al elemento <code><body></code> de la página web

Tabla 1.4. *Diversas propiedades para seleccionar elementos por tipo*